API SECURITY ASSESSMENT REPORT

BY KIMBERLY FARAI MAGO

FUTURE INTERNS | JSONPlaceholder

SUMMARY

This report presents the findings of a security assessment conducted on a public API using Postman and browser developer tools. The purpose of this assessment was to evaluate how the API handles user data, access control, and authentication mechanisms.

The API tested was:

- **JSONPlaceholder**

Testing focused on identifying vulnerabilities that could expose sensitive data or allow unauthorized access if this API were deployed in a real-world production environment.

OBJECTIVE

The objective of this assessment was to:

- Identify security vulnerabilities in API endpoints
- Evaluate authentication and authorization mechanisms
- Determine whether sensitive data is exposed
- Assess risks from a business and user privacy perspective

METHODOLOGY

The API was tested using the following tools and techniques:

- **Tools Used:**
  - Postman — to send HTTP requests and analyze responses
  - Browser Developer Tools — to inspect network traffic and headers

ENDPOINTS TESTED

The following API endpoints were tested using GET requests:

/users
/posts
/comments

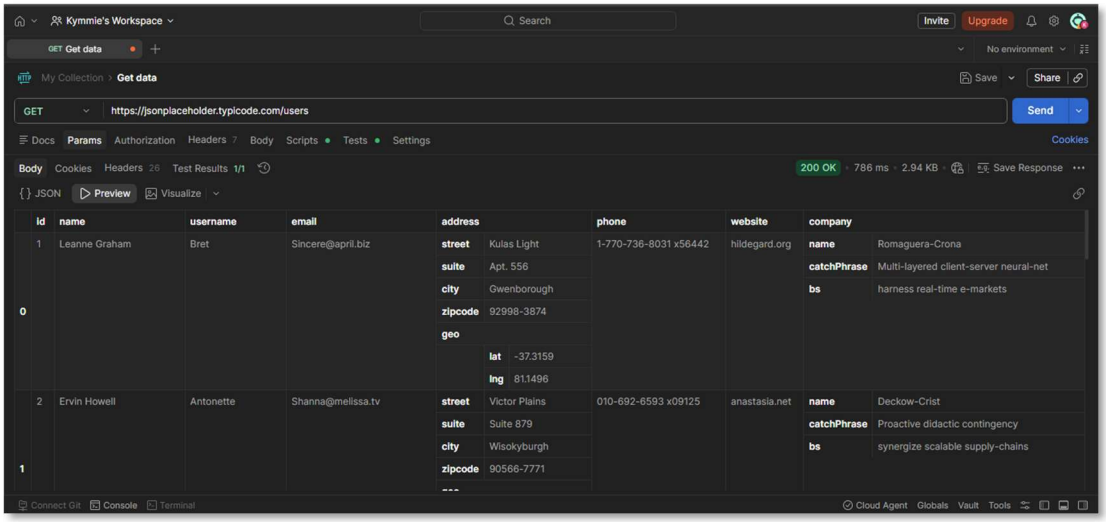
TESTING APPROACH

- **Testing Approach**
  - Sent requests without authentication
  - Observed response data
  - Checked headers for security controls
  - Verified access via browser (DevTools)
  - Analyzed data exposure and structure

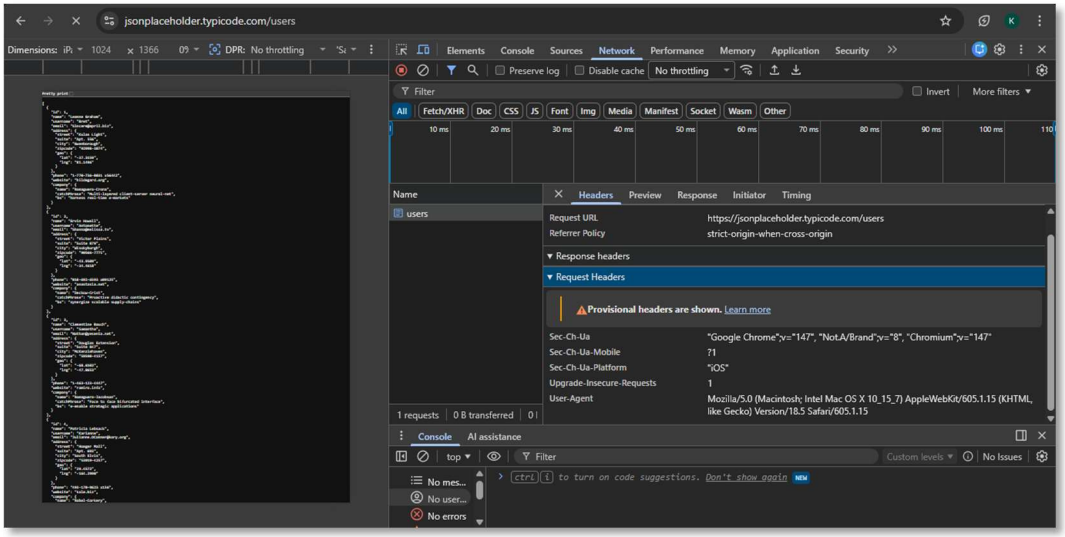
EVIDENCE & FINDINGS

FINDING 1:

UNAUTHENTICATED ACCESS TO SENSITIVE USER DATA



➢ Postman screenshot of `/users`



➢ DevTools screenshot

Explanation

The `/users` endpoint returns detailed user information including:

- Full names
- Email addresses
- Phone numbers
- Physical addresses
- Geographic coordinates (latitude & longitude)
- Company information

This data is accessible **without any authentication**, API key, or authorization mechanism.

From your DevTools screenshot:

- Request URL is visible
- No Authorization header is present
- Response returns full dataset

This confirms that the API is publicly accessible.

Business Impact

This is a **critical vulnerability** because:

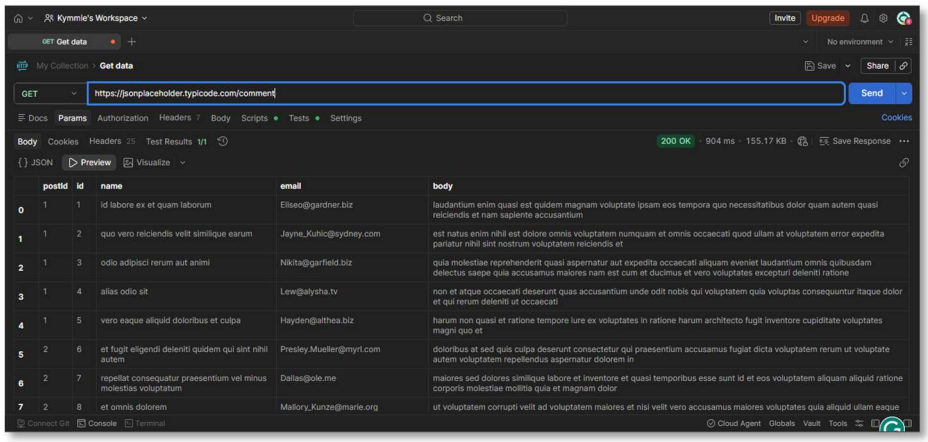
- Sensitive personal data is exposed publicly
- Violates privacy regulations, for example, POPIA.
- Enables attackers to:
    - Perform identity theft
    - Launch phishing attacks
    - Conduct social engineering

Risk Level: HIGH

Recommendation

- Implement authentication (API keys or OAuth 2.0)
- Restrict access to authorized users only
- Encrypt sensitive data

FINDING 2: EXPOSURE OF EMAIL ADDRESSES



➢ Postman screenshot of `/comments`

Explanation

The `/comments` endpoint exposes user email addresses such as:

`"email": "Eliseo@gardner.biz"`

These emails are publicly accessible without restrictions.

Business Impact

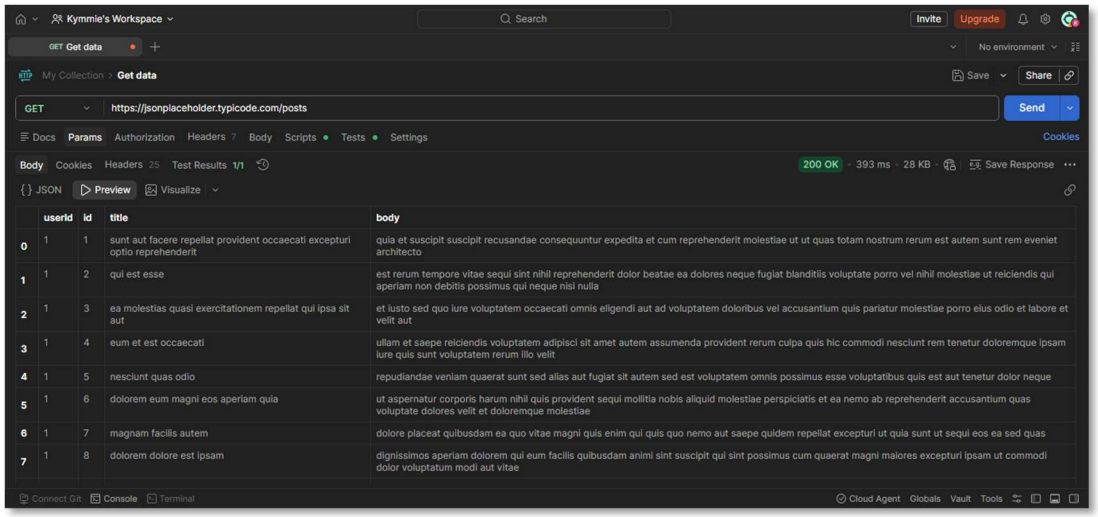
- Enables large-scale email harvesting
- Increases risk of phishing and spam campaigns
- Violates user privacy

Risk Level: HIGH

Recommendation

- Do not expose email addresses publicly
- Mask or anonymize sensitive fields
- Require authentication

FINDING 3: LACK OF ACCESS CONTROL



	userId	id	title	body
0	1	1	sunt aut facere repellat provident occaecati excepturi optio reprehenderit	quia et suscipit suscipit recusandae consequuntur expedita et cum reprehenderit molestiae ut ut quas totam nostrum rerum est autem sunt rem eveniet architecto
1	1	2	qui est esse	est rerum tempore vitae sequi sint nihil reprehenderit dolor beatae ea dolores neque fugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis qui aperiam non debitis possimus qui neque nisi nulla
2	1	3	ea molestias quasi exercitationem repellat qui ipsa sit aut	et iusto sed quo iure voluptatem occaecati omnis eligendi aut ad voluptatem doloribus vel accusantium quis pariatur molestiae porro eius odio et labore et velit aut
3	1	4	eum et est occaecati	ullam et saepe reiciendis voluptatem adipisci sit amet autem assumenda provident rerum culpa quis hic commodi nesciunt rem tenetur doloremque ipsam iure quis sunt voluptatem rerum illo velit
4	1	5	nesciunt quas odio	repudiandae veniam quaerat sunt sed alias aut fugiat sit autem sed est voluptatem omnis possimus esse voluptatibus quis est aut tenetur dolor neque
5	1	6	dolorem eum magni eos aperiam quia	ut aspernatur corporis harum nihil quis provident sequi mollitia nobis aliquid molestiae perspiciatis et ea nemo ab reprehenderit accusantium quas voluptate dolores velit et doloremque molestiae
6	1	7	magnam facilis autem	dolore placeat quibusdam ea quo vitae magni quis enim qui quis quo nemo aut saepe quidem repellat excepturi ut quia sunt ut sequi eos ea sed quas
7	1	8	dolorem dolore est ipsam	dignissimos aperiam dolorem qui eum facilis quibusdam animi sint suscipit qui sint possimus cum quaerat magni maiores excepturi ipsam ut commodi dolor voluptatum modi aut vitae

➢ Postman screenshot of `/posts`

Explanation

The `/posts` endpoint returns data from multiple users:

```
"userId": 1
"userId": 2
"userId": 3
```

There are no restrictions on which user can access which data.

Business Impact

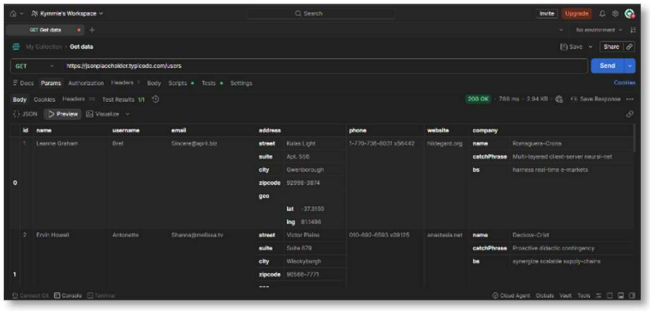
- Users can access other users' data
- Violates data isolation principles
- Can lead to data breaches

Risk Level: HIGH

Recommendation

- Implement role-based access control (RBAC)
- Validate user permissions before returning data

FINDING 4: EXCESSIVE DATA EXPOSURE



➢ Postman screenshot of `/users`

Explanation

The API returns more data than necessary, including:

- Full addresses
- Geo-location coordinates
- Company details

A typical application may only require minimal fields, for example, name and username.

Business Impact

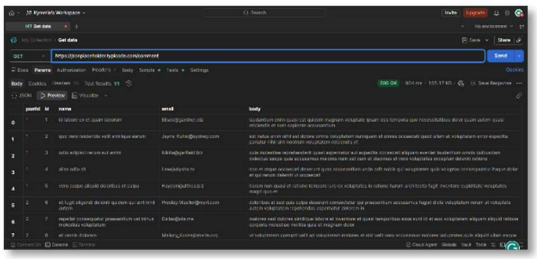
- Increased risk of data leakage
- Expands attack surface
- Unnecessary exposure of internal data

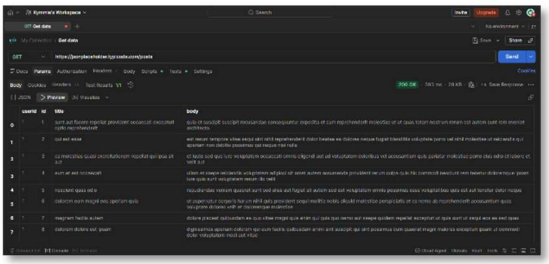
Risk Level: MEDIUM

Recommendation

- Implement data minimization
- Return only required fields
- Use filtering mechanisms

FINDING 5: LACK OF PAGINATION





- Postman screenshots of `/comments` and `/posts`

Explanation

The API returns large datasets, for example, 100 records in a single request without pagination.

Business Impact

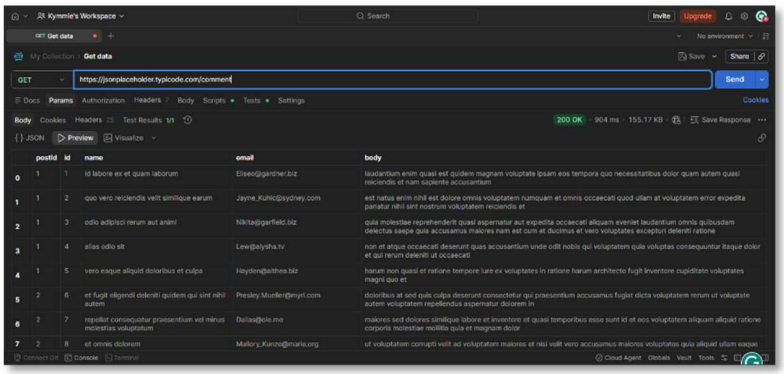
- Enables automated data scraping
- Can overload servers
- Impacts performance

Risk Level: MEDIUM

Recommendation

- Implement pagination (`limit`, `offset`)
- Apply rate limiting

FINDING 6: BROKEN OBJECT-LEVEL AUTHORIZATION



- Postman screenshot of `/comments`

Explanation

The API allows access to comments belonging to different posts without verifying user permissions.

Business Impact

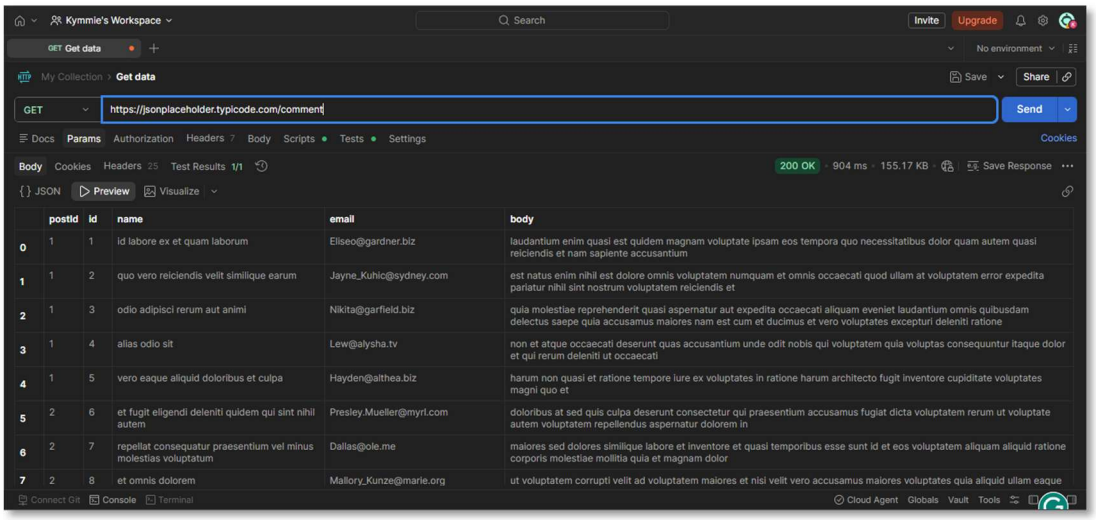
- Unauthorized access to other users data
- One of the most critical API vulnerabilities (OWASP API Top 10)

Risk Level: HIGH

Recommendation

- Validate ownership before returning data
- Enforce strict authorization checks

FINDING 7: PREDICTABLE RESOURCE ID'S



➢ Postman screenshot of `/comments`

Explanation

Sequential IDs allow attackers to easily guess and retrieve data.

Business Impact

- Enables data scraping
- Makes enumeration attacks easy

Risk Level: MEDIUM

Recommendation

- Use UUIDs instead of sequential IDs
- Combine with authorization controls

CONCLUSION

The API demonstrates multiple critical security vulnerabilities across all tested endpoints (`/users`, `/posts`, `/comments`). The most significant issues include lack of authentication, excessive data exposure, and broken access control mechanisms.

If this API were deployed in a production environment, it could result in severe data breaches, privacy violations, and reputational damage to the organization.

RECOMMENDATIONS

- Implement authentication (API keys, OAuth 2.0)
- Enforce proper authorization controls
- Limit data exposure (data minimization)
- Introduce pagination and rate limiting
- Replace sequential IDs with secure identifiers